

# AMAZON CLONE

## Full-Stack Architecture & Team Guide

React + Vite · Node.js + Express · MongoDB Atlas · JWT Auth

Version 1.0 · 6-Student Team · April 2026

### PROJECT OVERVIEW

|                  |                                                      |
|------------------|------------------------------------------------------|
| <b>Project</b>   | Amazon Clone — E-commerce Web Application            |
| <b>Stack</b>     | React (Vite) + Node.js/Express + MongoDB Atlas       |
| <b>Auth</b>      | JWT (JSON Web Tokens) + bcrypt password hashing      |
| <b>Team Size</b> | 6 Students — 3 Frontend, 3 Backend                   |
| <b>Duration</b>  | 3–4 Hours (Hackathon Style)                          |
| <b>Goal</b>      | Working app: Login → Products → Cart → Checkout flow |

### SYSTEM ARCHITECTURE

| BROWSER / CLIENT            |                    | BACKEND SERVER                   |                    | DATABASE                 |
|-----------------------------|--------------------|----------------------------------|--------------------|--------------------------|
| React + Vite<br>(Port 5173) | ■■■■■■■<br>■■■■■■■ | Node.js + Express<br>(Port 5000) | ■■■■■■■<br>■■■■■■■ | MongoDB Atlas<br>(Cloud) |
| Axios HTTP calls            |                    | REST APIs + JWT                  |                    | Collections              |

■ The Frontend (React) runs on port 5173. The Backend (Express) runs on port 5000. They communicate via HTTP. MongoDB is hosted on the cloud — no local install needed.

### TECH STACK — FULL DETAILS

| Layer    | Technology       | Version | Purpose                                |
|----------|------------------|---------|----------------------------------------|
| Frontend | React            | 18.x    | UI component library                   |
| Frontend | Vite             | 5.x     | Dev server & bundler (faster than CRA) |
| Frontend | React Router DOM | 6.x     | Client-side page routing               |
| Frontend | Axios            | 1.x     | HTTP requests to backend               |

|                  |                          |         |                                   |
|------------------|--------------------------|---------|-----------------------------------|
| <b>Frontend</b>  | CSS / Tailwind           | —       | Styling                           |
| <b>Backend</b>   | Node.js                  | 18+ LTS | JavaScript runtime                |
| <b>Backend</b>   | Express.js               | 4.x     | REST API framework                |
| <b>Backend</b>   | MongoDB Atlas            | Cloud   | NoSQL cloud database              |
| <b>Backend</b>   | Mongoose                 | 7.x     | MongoDB ODM / schema layer        |
| <b>Auth</b>      | bcryptjs                 | 2.x     | Password hashing                  |
| <b>Auth</b>      | jsonwebtoken             | 9.x     | JWT token creation & verification |
| <b>Auth</b>      | dotenv                   | 16.x    | Environment variables (.env file) |
| <b>Dev Tools</b> | Postman / Thunder Client | —       | API testing                       |
| <b>Dev Tools</b> | VS Code                  | —       | Code editor                       |
| <b>Dev Tools</b> | GitHub                   | —       | Version control & collaboration   |

## ■ FOLDER STRUCTURE (Everyone Must Follow)

### Frontend — amazon-frontend/

```
amazon-frontend/
  ■■■■ src/
  ■ ■■■■ components/
  ■ ■ ■■■■ Navbar.jsx ← shared by ALL
  ■ ■■■■ pages/
  ■ ■ ■■■■ ProductList.jsx ← Student 1
  ■ ■ ■■■■ ProductDetail.jsx ← Student 2
  ■ ■ ■■■■ Cart.jsx ← Student 3
  ■ ■ ■■■■ Login.jsx ← Student 4 (you)
  ■ ■ ■■■■ Signup.jsx ← Student 4 (you)
  ■ ■■■■ context/
  ■ ■ ■■■■ AuthContext.jsx ← Student 4 creates this
  ■ ■■■■ App.jsx ← Routes – ALL contribute
  ■ ■■■■ main.jsx
  ■■■■ .env ← VITE_API_URL=http://localhost:5000
  ■■■■ package.json
```

### Backend — amazon-backend/

```
amazon-backend/
  ■■■■ models/
  ■ ■■■■ User.js ← Student 4 (you)
  ■ ■■■■ Product.js ← Student 5
  ■ ■■■■ Cart.js ← Student 6
  ■■■■ routes/
  ■ ■■■■ auth.js ← Student 4 (you)
  ■ ■■■■ products.js ← Student 5
  ■ ■■■■ cart.js ← Student 6
  ■■■■ middleware/
  ■ ■■■■ authMiddleware.js ← Student 4 creates – others use it
  ■■■■ .env ← MONGO_URI + JWT_SECRET
  ■■■■ server.js ← Main entry point
  ■■■■ package.json
```

## ■ DATABASE SCHEMAS (MongoDB)

| <b>Collection</b> | <b>Fields</b>                                                                                                            | <b>Notes</b>                                                       |
|-------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| users             | name (String, required)<br>email (String, unique, required)<br>password (String, hashed)<br>createdAt (Date)             | Password is NEVER stored plain.<br>Bcrypt hashes it before saving. |
| products          | title (String, required)<br>price (Number, required)<br>image (String, URL)<br>description (String)<br>category (String) | Seeded by Student 5.<br>All other students read from this.         |

|       |                                                                                                                   |                                               |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| carts | userId (ObjectId → ref users)<br>items: [{<br>productId (ObjectId)<br>quantity (Number)<br>}]<br>updatedAt (Date) | One cart per user.<br>Student 6 manages this. |
|-------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|

## ■ COMPLETE API REFERENCE

### ■ Authentication APIs — Student 4 (YOU)

| <b>Method</b> | <b>Endpoint</b>  | <b>Body</b>               | <b>Response</b>          | <b>Auth Required</b> |
|---------------|------------------|---------------------------|--------------------------|----------------------|
| POST          | /api/auth/signup | { name, email, password } | { message, token, user } | No                   |
| POST          | /api/auth/login  | { email, password }       | { token, user }          | No                   |
| GET           | /api/auth/me     | — (header only)           | { user object }          | Yes (JWT)            |

### ■ Product APIs — Student 5

| <b>Method</b> | <b>Endpoint</b>   | <b>Body</b>                          | <b>Response</b>       | <b>Auth Required</b> |
|---------------|-------------------|--------------------------------------|-----------------------|----------------------|
| GET           | /api/products     | —                                    | [ array of products ] | No                   |
| GET           | /api/products/:id | —                                    | { single product }    | No                   |
| POST          | /api/products     | { title, price, image, description } | { created product }   | Yes (Admin)          |

### ■ Cart APIs — Student 6

| <b>Method</b> | <b>Endpoint</b>   | <b>Body</b>             | <b>Response</b>     | <b>Auth Required</b> |
|---------------|-------------------|-------------------------|---------------------|----------------------|
| GET           | /api/cart         | —                       | { cart with items } | Yes (JWT)            |
| POST          | /api/cart         | { productId, quantity } | { updated cart }    | Yes (JWT)            |
| PUT           | /api/cart/:itemId | { quantity }            | { updated cart }    | Yes (JWT)            |
| DELETE        | /api/cart/:itemId | —                       | { updated cart }    | Yes (JWT)            |

■ Cart and /me APIs require a JWT token. Send it in the request header as: Authorization: Bearer — Student 4 provides the token after login.

## ■ TEAM ASSIGNMENTS — WHO DOES WHAT

### Student 1      Frontend — Product Listing Page

Student 1

#### Tasks:

GET /api/products

- Create ProductList.jsx with product cards
- Display: image, title, price using .map()
- Fetch from GET /api/products using axios
- Add 'View Details' button linking to /product/:id
- Basic CSS grid/flex layout

### Student 2      Frontend — Product Detail Page

Student 2

#### Tasks:

GET /api/products/:id

- Create ProductDetail.jsx
- Read product ID from URL params (useParams)
- Fetch from GET /api/products/:id
- Show: image, title, price, description
- Add 'Add to Cart' button → POST /api/cart

### Student 3      Frontend — Cart Page

Student 3

#### Tasks:

GET/PUT/DELETE /api/cart

- Create Cart.jsx
- Fetch cart from GET /api/cart with JWT
- Show items with image, name, price, quantity
- Add +/- quantity buttons → PUT /api/cart/:id
- Remove button → DELETE /api/cart/:id

### Student 4      Backend — Auth APIs + Frontend Login/Signup

YOU

**Tasks:**

POST /signup, /login | GET /me

- Create User.js Mongoose model
- POST /api/auth/signup — hash password with bcrypt, return JWT
- POST /api/auth/login — compare password, return JWT
- GET /api/auth/me — protected, return logged-in user
- Create authMiddleware.js — verifies JWT (used by Students 3 & 6)
- Create Login.jsx and Signup.jsx pages
- Create AuthContext.jsx to store token globally

**Student 5****Backend — Product APIs**

Student 5

**Tasks:**

GET /products, /products/:id

- Create Product.js Mongoose model
- GET /api/products — return all products
- GET /api/products/:id — return one product
- Seed 5–8 dummy products into MongoDB

**Student 6****Backend — Cart APIs**

Student 6

**Tasks:**

POST/PUT/DELETE /cart

- Create Cart.js Mongoose model (userId + items[])
- POST /api/cart — add item to cart (protected)
- PUT /api/cart/:itemId — update quantity (protected)
- DELETE /api/cart/:itemId — remove item (protected)
- Use authMiddleware from Student 4

## ■ YOUR PART IN DETAIL — Authentication (Student 4)

■ You are the most important student on the team. Your JWT middleware protects ALL sensitive routes. Students 3 and 6 CANNOT finish without your authMiddleware.js. Finish your backend FIRST.

### Step 1 — Install Packages

```
cd amazon-backend
npm install express mongoose bcryptjs jsonwebtoken dotenv cors
```

### Step 2 — .env file (NEVER commit this!)

```
MONGO_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/amazonclone
JWT_SECRET=supersecretkey123
PORT=5000
```

### Step 3 — User Model (models/User.js)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

### Step 4 — Auth Routes (routes/auth.js)



```

const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const auth = require('../middleware/authMiddleware');
const router = express.Router();

// SIGNUP
router.post('/signup', async (req, res) => {
  const { name, email, password } = req.body;
  const hashed = await bcrypt.hash(password, 10);
  const user = await User.create({ name, email, password: hashed });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
  res.json({ token, user: { id: user._id, name, email } });
});

// LOGIN
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user) return res.status(400).json({ error: 'User not found' });
  const match = await bcrypt.compare(password, user.password);
  if (!match) return res.status(400).json({ error: 'Wrong password' });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
  res.json({ token, user: { id: user._id, name: user.name, email } });
});

// GET CURRENT USER (protected)
router.get('/me', auth, async (req, res) => {
  const user = await User.findById(req.user.id).select('-password');
  res.json(user);
});

module.exports = router;

```

## Step 5 — Auth Middleware (middleware/authMiddleware.js)

■ Share this file with Students 3 and 6 — they import it to protect cart routes.

```

const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const header = req.headers.authorization;
  if (!header) return res.status(401).json({ error: 'No token' });
  const token = header.split(' ')[1]; // 'Bearer '
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // { id: userId }
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
};

```

## Step 6 — Frontend: AuthContext.jsx

```
import { createContext, useContext, useState } from 'react';
const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [token, setToken] = useState(localStorage.getItem('token'));
  const [user, setUser] = useState(null);

  const login = (tkn, usr) => {
    localStorage.setItem('token', tkn);
    setToken(tkn); setUser(usr);
  };
  const logout = () => {
    localStorage.removeItem('token');
    setToken(null); setUser(null);
  };

  return (
    <AuthContext.Provider value={{ token, user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

## ■ HOW OTHER STUDENTS USE YOUR JWT TOKEN

Students 3 and 6 need to send the token in every protected request:

```
import axios from 'axios';
import { useAuth } from '../context/AuthContext';

function CartPage() {
  const { token } = useAuth(); // Get token from your AuthContext

  const fetchCart = async () => {
    const res = await axios.get('http://localhost:5000/api/cart', {
      headers: { Authorization: `Bearer ${token}` }
    });
    console.log(res.data);
  };
}
```

### App.jsx — Routes Configuration (team agreement)

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import ProductList from './pages/ProductList'; // Student 1
import ProductDetail from './pages/ProductDetail'; // Student 2
import Cart from './pages/Cart'; // Student 3
import Login from './pages/Login'; // Student 4 (you)
import Signup from './pages/Signup'; // Student 4 (you)

export default function App() {
  return (
    <AuthProvider>
    <BrowserRouter>
    <Routes>
    <Route path="/" element={<ProductList />} />
    <Route path="/product/:id" element={<ProductDetail />} />
    <Route path="/cart" element={<Cart />} />
    <Route path="/login" element={<Login />} />
    <Route path="/signup" element={<Signup />} />
    </Routes>
    </BrowserRouter>
    </AuthProvider>
  );
}
```

## ■ FIRST-TIME SETUP GUIDE (Do This Together)

1

### Clone/Create Repos

Create two folders: amazon-frontend and amazon-backend Run: git init in each. Create one GitHub repo per project.

2

### Frontend Setup

npm create vite@latest amazon-frontend -- --template react cd amazon-frontend && npm install npm install axios react-router-dom

3

### Backend Setup

mkdir amazon-backend && cd amazon-backend && npm init -y npm install express mongoose bcryptjs jsonwebtoken dotenv cors

4

### MongoDB Atlas

Go to [mongodb.com/cloud/atlas](https://mongodb.com/cloud/atlas) → Free cluster → Connect Copy connection string → paste in backend .env as MONGO\_URI

5

### Start Servers

Frontend: npm run dev (runs on <http://localhost:5173>) Backend: node server.js (runs on <http://localhost:5000>)

6

### Enable CORS (very important!)

In server.js: `app.use(cors({ origin: 'http://localhost:5173' })))` Without this, frontend cannot talk to backend.

## COMMON PROBLEMS & FIXES

| Problem                                | Fix                                                                                                                        |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>CORS Error</b>                      | Add cors middleware in server.js BEFORE routes.<br>app.use(cors({ origin: 'http://localhost:5173' })))                     |
| <b>401 Unauthorized</b>                | JWT token missing or expired. Check Authorization header.<br>Format: Bearer <token> (with space after Bearer)              |
| <b>Cannot connect to MongoDB</b>       | Check MONGO_URI in .env — it must have your actual password.<br>Also whitelist 0.0.0.0/0 in Atlas Network Access settings. |
| <b>Module not found</b>                | Run npm install again. Check spelling of package name.                                                                     |
| <b>Port already in use</b>             | Kill the process: npx kill-port 5000 (or 5173)                                                                             |
| <b>Password stored as plain text</b>   | Always hash with bcrypt BEFORE saving. NEVER save req.body.password directly.                                              |
| <b>Token not persisting on refresh</b> | Save token in localStorage after login. AuthContext reads from localStorage on load.                                       |
| <b>React state not updating</b>        | setState is async — don't read state right after setting. Use useEffect.                                                   |

## FINAL DELIVERABLE CHECKLIST

| Student                | Must Demo                                                     | Done? |
|------------------------|---------------------------------------------------------------|-------|
| Student 1              | Product list page shows cards fetched from API                | ■     |
| Student 2              | Clicking a product shows detail page + Add to Cart works      | ■     |
| Student 3              | Cart shows items, can update quantity & remove                | ■     |
| <b>Student 4 (You)</b> | Signup creates user, Login returns token, protected /me works | ■     |
| Student 5              | GET /products returns JSON list, GET /products/:id works      | ■     |
| Student 6              | POST/PUT/DELETE /cart works with JWT token                    | ■     |
| All Together           | Full flow: Signup → Login → Browse → Add to Cart → View Cart  | ■     |

*"You are not coding. You are solving problems using AI."*